

MGMT298D

Science and Strategy of AI

Week 8

Building LLM Apps

UCLA Anderson School of Management

Auyon Siddiq

Week 7 Recap: LLMs in the Wild

Pre-training learns broad language patterns from huge text corpora

Post-training turns a base model into a useful assistant

Inference is the product moment: user input → model output

This week: how do we wrap that model in an app?

Today's Class

Part 1: Deployment approaches

Prompting, RAG, and fine-tuning — three ways to adapt an LLM

Part 2: App project + toolkit

How your team will build, test, and deploy a working product

Part 1: Deployment Approaches

The Builder's Question

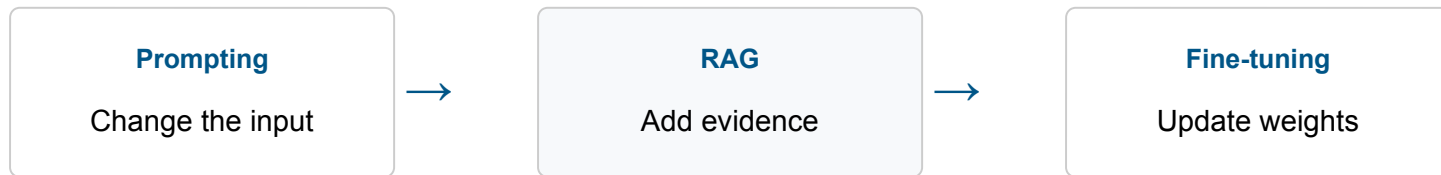
Same frontier model, many possible products

The model is only one component — prompts, data, UI, and workflow matter

Builder's question: what should change when the app needs better answers?

Input? Evidence? Weights?

Three Ways to Adapt an LLM



All three start from the same base model.

Most real apps combine at least two.

Which Lever Solves Which Problem?

Prompting

Need better behavior

Need clearer format

Need faster iteration

Change instructions first

RAG

Need fresh facts

Need private documents

Need citations

Bring evidence at runtime

Fine-tuning

Need stable style

Need narrow skill

Need cheaper model

Train on examples

Approach 1: Prompting

Same Weights, Different Prompt

Prompting changes the input, not the model weights

Bare prompt: short, underspecified, easy to misread

Better prompt: role + task + context + format

Most app behavior starts here

What Goes Into a Prompt?



Good prompts make the desired behavior boringly explicit.

If the app needs consistency, specify the contract.

System Prompts and Roles

System message: high-priority instruction for the assistant

User message: what the person asks right now

Developer message: app-level rules and workflow constraints

Prompt hierarchy is product design, not just wording

Few-Shot Prompting

Few-shot prompting: show examples in the prompt

The model imitates the pattern at inference time

Useful for classification, rewriting, extraction, and formatting

No training step — just better examples

Prompting: When It Works

Use prompting when...	Why it fits
Behavior is simple	Instructions are enough
Iteration matters	Change text, test again
Data is not private	No retrieval layer needed
Output must follow a format	Examples anchor the pattern
You are still exploring	Cheapest lever to adjust

Start with prompting. It is the fastest honest test.

Approach 2: RAG

RAG Adds Evidence at Runtime

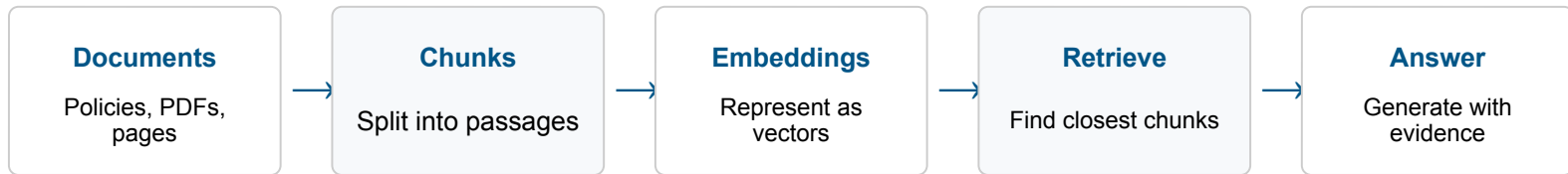
RAG = retrieval-augmented generation

Retrieve relevant chunks from your documents

Add those chunks to the prompt before the model answers

The model still generates — it just has better context

RAG Architecture



Indexing happens offline. Retrieval happens for each user question.

The app decides what evidence reaches the model.

Chunking Is a Design Choice

Small chunks are precise, but may lose surrounding context

Large chunks preserve context, but can retrieve extra noise

Overlap helps avoid splitting an idea in half

Retrieval quality starts before the model sees anything

RAG Failure Modes

Wrong retrieval: the answer never had the right evidence

Weak synthesis: evidence was present, but the model used it badly

Stale index: source documents changed, embeddings did not

RAG reduces hallucination. It does not eliminate it.

RAG: When It Works

Use RAG when...	Why it fits
Facts change often	Update the index, not the weights
Knowledge is private	Retrieve from your own corpus
Users need citations	Point back to source chunks
Coverage is broad	Store more than fits in the context window
Answers need grounding	Evidence travels with the prompt

If the problem is missing knowledge, retrieve before you train.

Approach 3: Fine-Tuning

Fine-Tuning Updates Weights

Fine-tuning changes the model, not just the prompt

Builder version usually means supervised fine-tuning (SFT)

Training examples look like: prompt → ideal answer

Good for repeated behavior you want baked in

What Fine-Tuning Looks Like

Method

Supervised fine-tuning on prompt / ideal-answer pairs

Data

Hundreds to a few thousand high-quality examples

Target

Consistent style, format, tone, or narrow domain behavior

Risk

Bad examples teach the model bad habits very efficiently

Fine-Tuning: When It Works

Use fine-tuning when...	Why it fits
Behavior repeats	Training examples can cover it
Style must be stable	Weights learn the pattern
Prompt is too long	Reduce per-call instruction overhead
Smaller model is enough	Specialize cheaper infrastructure
You can evaluate outputs	Training without eval is guesswork

Fine-tuning is powerful, but unforgiving.

The Three Levers Compared

Question	Prompting	RAG	Fine-tuning
Changes weights?	No	No	Yes
Adds new facts?	A little	Yes	Sometimes
Setup cost	Hours	Days to weeks	Weeks+
Best for	Behavior	Knowledge	Stable skill
Debug by	Inspect prompt	Inspect retrieval	Inspect examples
First move?	Usually	When facts matter	After evals

Matching the lever to the failure mode is the whole game.

A Simple Debugging Rule

Bad format? Improve the prompt.

Missing or stale fact? Add retrieval.

Inconsistent repeated behavior? Consider fine-tuning.

Do not train your way out of a product problem.

15-min Break

z z z



Part 2: App Project + Toolkit

App Project: The Goal

Build a web app that uses an LLM to solve a specific problem

The app should feel like a product, not a prompt pasted into a page

AI should make the workflow meaningfully better

Start from the user's painful task

Teams and Deliverable

Teams: groups of 3 students

Deliverable: public app URL + short project write-up

Live date: 11:59pm PT, May 27

Final presentations: Week 10

Picking a Use Case

What task is painful, repetitive, or judgment-heavy?

What information does the user bring?

What does the app return that is actually useful?

If AI disappeared, would the product still make sense?

Good Starter App Shapes

Advisor

Tutor

Coach

Decision support

Helps a user think

Analyst

Summarizer

Extractor

Classifier

Turns input into structure

Workflow

Draft

Review

Revise

Moves work forward

Development Toolkit

Node.js + Express for the small web server

Plain HTML is enough; React is optional

Gemini API for the LLM call

Railway for deployment

AI Coding With Cowork / Codex

You do not need to write every line manually

Use the assistant to inspect folders, edit files, and debug errors

Ask it to explain changes before you accept them

Good builder habit: small steps, frequent tests

Getting Your Gemini API Key

- 1 Open AI Studio** Go to aistudio.google.com
- 2 Sign in** Use your Google account
- 3 Create key** Click Get API Key, then create a new key
- 4 Store it** Put the key in `.env`, never in public code

Secrets belong in environment variables.

Deploying on Railway

- 1 Create account** Sign up at railway.com
- 2 Install CLI** `brew install railway` or `npm install -g @railway/cli`
- 3 Login** Run `railway login` from the terminal
- 4 Deploy** Connect the folder, set variables, publish

Local app first. Public URL second.

Setup Checklist

Tool	Why we need it	Quick check
Node.js	Run the server	node --version
Gemini key	Call the model	.env contains API key
Railway	Deploy the app	railway login
GitHub	Share code if needed	repo exists
Lab kit	Starter instructions	README is open

Ask the most technical teammate to lead setup.

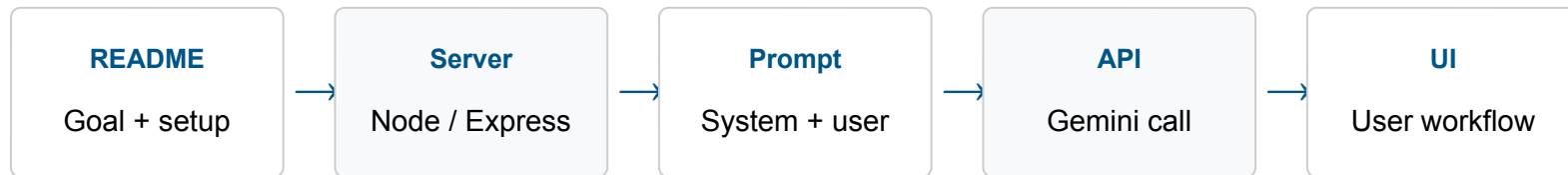
Work Session

Today's Work Session

- 1 Form team** Groups of 3. Write names down.
- 2 Pitch ideas** Each person brings 2 possible use cases.
- 3 Pick one** Choose the painful task, not the flashiest demo.
- 4 Define value** Why does AI make this better?
- 5 Open kit** Start the folder and first README.

Leave class with a real product direction.

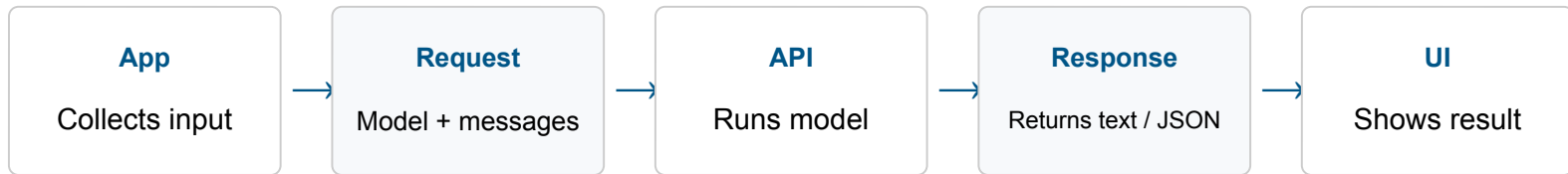
How the App Fits Together



One project folder holds the instructions, code, prompts, and deployment settings.

Cowork / Codex can work inside that folder with you.

How an LLM API Call Works



An API is a contract: send structured input, receive structured output.

The API key proves your app is allowed to call the model.

What APIs Let Builders Do

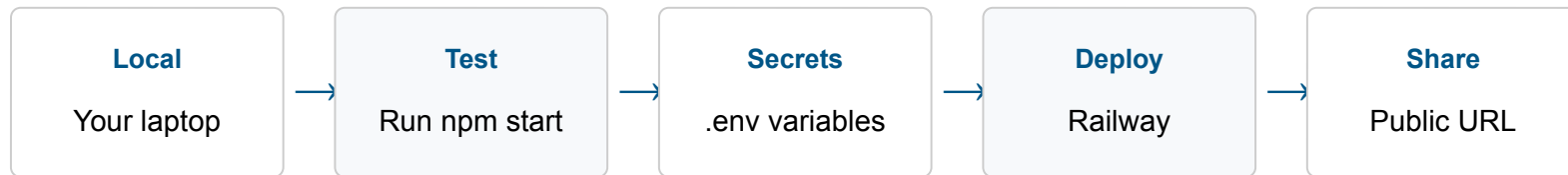
Put an LLM behind a button, form, chat box, or workflow

Ask for text, structured JSON, classification, or extraction

Control temperature, max tokens, and system instructions

Treat frontier AI like a software component

From Local Folder to Public URL



Same project, different environment.

If it does not run locally, deployment will not magically fix it.

What Makes a Good Final App?

Clear user: someone specific has a real task

Clear workflow: input, AI step, useful output

Clear improvement: faster, better, cheaper, or more personalized

Clear limits: what the app should not do

Before Next Class

Team formed

Use case selected

Gemini key created and stored in .env

Starter app runs locally

Come ready to build

Glossary (1/2)

Prompting	Changing the model input to steer behavior without updating weights.
System prompt	High-priority instruction that defines role, rules, and boundaries.
Few-shot	Including examples in the prompt so the model imitates a pattern.
RAG	Retrieval-augmented generation; adding retrieved evidence before generation.
Chunk	A passage-sized piece of a document stored for retrieval.
Embedding	Numeric representation used to compare semantic similarity.
Vector DB	Database optimized for nearest-neighbor search over embeddings.

Glossary (2/2)

Fine-tuning	Updating model weights using additional training examples.
SFT	Supervised fine-tuning on prompt / ideal-answer pairs.
API	Contract for sending structured requests and receiving structured responses.
Endpoint	Specific URL where an API receives requests.
Environment variable	Secret or config value stored outside source code.
Deployment	Moving an app from local development to a public server.
Railway	Cloud platform we will use to host project apps.